

Application de Livraison — SafeLogistic

Attention ! Avant de commencer à programmer, prenez le temps de lire l'ensemble du sujet et de modéliser votre problème. Les noms de classes sont donnés à titre indicatif : vous êtes libres de les modifier.

Objectifs Pédagogiques Clés

Ce TP a pour objectif de faire le lien entre les notions d'abstraction vu en cours, ainsi que de faire le pont entre la théorie du TD 3 et mettre un cadre applicatif pour le TP2 en exploitant :

- Classes Abstraites & Interfaces (TD 3, Sec. 1, 2, 3) : Pour étendre les notions de Graph, Node, et Algorithm et les rendre spécifiques à la logistique.
- Héritage & Polymorphisme (TD 3, Sec. 1) : Pour créer des types de sommets spécialisés (Restaurant, Client, Depot) qui héritent d'une classe de base Node du TP2. Composition (TD 3, Sec. 2) : Pour la structure du graphe et la gestion du manager de livraison.
- Encapsulation et Membres de Classe (TD 3, Sec. 4) : Dans l'implémentation de l'algorithme de Dijkstra (par exemple, utilisation de variables de classe static pour des compteurs ou des constantes globales).

Contexte : SafeLogistic

Imaginez que vous travaillez pour SafeLogistic, une entreprise qui souhaite mettre en place un système de livraison de repas performant, sécurisé et transparent. Pour ce faire, elle doit concevoir un logiciel qui permet à l'ensemble des protagonistes d'une livraison (livreurs, clients, restaurateurs, etc.) d'avoir un accès aux informations propres à leur statut et à leur position dans la chaîne logistique. Le système tend à satisfaire l'ensemble des acteurs en mettant en place des algorithmes dédiés pour optimiser les trajets et les délais.

Vous êtes en charge de concevoir ce système, ce qui nécessitera l'exploitation et l'extension de votre librairie de graphes (TP2) grâce aux concepts d'abstraction (TD3). Votre mission est d'implémenter ce système en suivant les étapes suivantes :

Étape 1 - Modélisation et Modularité du Réseau : Application de l'héritage et des interfaces pour spécialiser les nœuds et les arêtes (Restaurant, Client, arêtes pondérées).

Étape 2 - Implémentation des Algorithmes d'Efficiencia : Intégration de l'algorithme de Dijkstra pour le calcul des plus courts chemins.

Étape 3 - Gestion de la Logistique : Mise en place du **DeliveryManager** via la composition pour simuler la planification d'une commande complète.

Partie 1 : Modélisation du Réseau Logistique et Abstraction

Pour que le logiciel SafeLogistic soit performant, sécurisé et évolutif, la première étape est de modéliser le réseau de la ville en s'appuyant sur les principes d'abstraction et de modularité de la POO. Il s'agit d'intégrer des informations du monde réel à la librairie de graphes développée lors du TP2.

Spécialisation des Acteurs : Héritage et Polymorphisme

Dans le contexte de livraison, un point d'intérêt dans la ville doit représenter un acteur spécifique (Restaurant, Client, Dépôt). Ces acteurs partagent une nature commune (Node/Lieu) mais possèdent des comportements et des informations qui leur sont propres (leur statut).

Écho au TD 3, Section 1.1 (Création des Animaux par Héritage) : Nous allons spécialiser le concept de Node de la même manière que les Animals ont été spécialisés.

Lignes Directrices de Modélisation (Héritage) :

1. Abstraction du Lieu : Votre modélisation doit utiliser l'héritage pour définir un concept abstrait de Lieu (potentiellement via une classe abstraite `LocationNode` ou en modifiant Node du TP2).
2. Spécialisation des Types : Vous devez créer les classes concrètes nécessaires (`Restaurant`, `Client`, `Dépôt`) qui hériteront de ce concept abstrait de Lieu.
→ Chaque classe spécialisée devra intégrer les attributs nécessaires pour son rôle dans la livraison (ex : temps de préparation pour un restaurant, adresse pour un client, etc.).

Justification (Document UML/Rapport) : Vous expliquerez en quoi l'héritage est le mécanisme idéal ici (relation "est un") et comment il permet de réutiliser le comportement de base des Nodes tout en offrant la spécialisation par polymorphisme.

Modélisation des Trajets

Pour calculer le chemin le plus efficace, les arêtes du graphe ne peuvent plus être de simples liaisons. Elles doivent représenter une distance ou un temps de trajet, c'est-à-dire une pondération. **SafeLogistic** doit s'assurer que tout trajet urbain respecte le "contrat" de la pondération.

Écho au TD 3, Section 3 (Interface `SystemElement`) : Une interface peut imposer la notion de poids, de la même manière que l'interface `SystemElement` imposait la méthode `open()`.

Lignes Directrices de Modélisation (Interfaces et Classes Abstraites) :

1. Définition du Contrat : Vous devez définir un contrat (via une interface) qui spécifie que tout trajet (Arête) est pondérable et doit pouvoir retourner un coût (temps ou distance). L'interface `WeightedEdge` est une solution possible, mais vous êtes libre de choisir le nom et les méthodes qui conviennent le mieux à votre design.
2. Implémentation du Trajet : Implémentez une classe concrète (par exemple `StreetSegment`) qui étend la classe de base `Edge` de votre librairie (TP2) et respecte ce contrat de pondération

Justification (Document UML/Rapport) : Vous expliquerez pourquoi l'utilisation d'une interface (le "contrat") pour la pondération est préférable à l'héritage direct dans ce cas. Expliquez comment cela garantit la flexibilité pour intégrer de futurs types de liaisons (pistes cyclables, voies rapides, etc.) sans casser la logique de l'algorithme de calcul des chemins.

Partie 2 : Implémentation des Algorithmes d'Efficiencia (Dijkstra)

Le succès de SafeLogistic repose sur la capacité à trouver les trajets les plus rapides et les plus courts. L'application des algorithmes de parcours simples (BFS/DFS) du TP2 n'est plus suffisante, car ils ne tiennent

pas compte des poids (weight) que vous avez introduits. Cette partie vise à implémenter l'algorithme de Dijkstra pour calculer les plus courts chemins dans votre graphe pondéré.

Conception Modulaire de l'Algorithme

Pour garantir l'extensibilité du logiciel, l'algorithme de Dijkstra doit s'intégrer de manière modulaire à votre librairie de graphes, suivant le principe de séparation des préoccupations.

Écho au TD 3, Sections 1.2 (Implémentation des actions) & 3 (Contrat d'interface) : Votre algorithme doit respecter le "contrat" de la librairie et potentiellement étendre une classe abstraite d'algorithme.

Lignes Directrices de Modélisation (Modèle Algorithmique) :

1. Intégration du Cadre : Concevez une classe `DijkstraAlgorithm` qui utilise le cadre d'abstraction des algorithmes défini lors du TP2 (si une telle classe abstraite ou interface existait, vous devez l'étendre ou l'implémenter ; sinon, vous la créez).
2. Gestion des Données Pondérées : L'algorithme doit pouvoir accéder et utiliser le poids (`getWeight()`) des arêtes (vos objets `StreetSegment`) pour son calcul.

Implémentation de Dijkstra

L'algorithme de Dijkstra est l'outil standard pour déterminer le chemin optimal dans un réseau pondéré par des coûts positifs (le temps de trajet ne peut être négatif).

Lignes Directrices d'Implémentation :

1. Implémentation : Implémentez la logique complète de l'algorithme de Dijkstra dans votre classe dédiée. L'algorithme doit retourner non seulement le coût total du plus court chemin, mais aussi la séquence de sommets qui constitue ce chemin.
2. Choix des Structures de Données : Vous êtes encouragés à utiliser les structures de données Java les plus appropriées (comme les collections ou les files de priorité) pour optimiser l'efficacité de l'algorithme ($O((|V| + |E|) \log |V|)$ avec une file de priorité, par exemple).

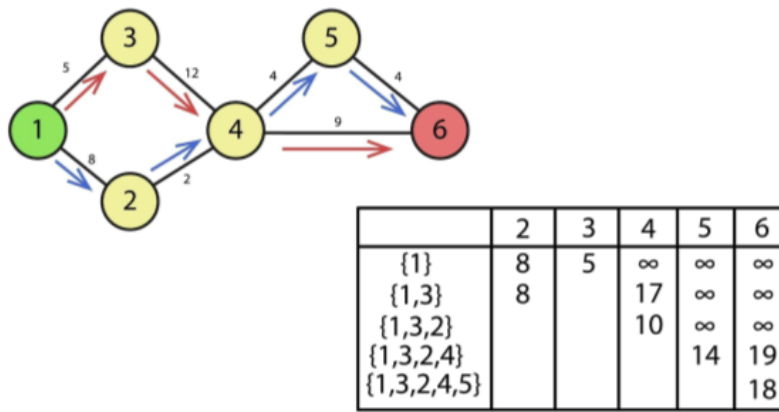
Justification (Document UML/Rapport) : Justifiez le choix de l'algorithme de Dijkstra plutôt qu'un autre algorithme de parcours (comme BFS/DFS ou Bellman-Ford) dans le contexte spécifique de SafeLogistic (graphe non orienté ou orienté, pondérations positives, recherche du chemin minimal).

Premiers Tests (Validation de l'Efficiency)

Afin de valider l'exactitude de l'algorithme, vous devez le tester sur un petit graphe de test que vous aurez vous-même créé.

Travail Demandé (Validation)

1. Jeu de Test Simple : Créez un petit graphe de test (par exemple, 5-6 lieux) à charger depuis un fichier, en vous assurant que les arêtes sont bien vos objets `StreetSegment` avec des poids.
2. Test de Chemin : Calculez le plus court chemin entre deux points éloignés de ce graphe et vérifiez manuellement que le résultat est correct.



Dijkstra's Algorithm

Partie 3 : Gestion de la Logistique et Composition

L'objectif final de SafeLogistic est d'exécuter une livraison. Cette partie consiste à créer un module applicatif (par opposition à la librairie du TP2) qui compose l'ensemble des éléments modélisés et algorithmiques développés lors des parties 1 et 2 pour simuler le processus complet. Ce module doit représenter le cœur du système de gestion des commandes.

Architecture du Manager de Livraison

Le système de SafeLogistic a besoin d'un point de contrôle central qui orchestre les actions (charger la ville, lancer l'algorithme, interpréter le résultat).

Écho au TD 3, Section 2 (Contes en Tiroir / Composition) : De même que le Conte se compose de plusieurs Parties (String ou d'autres Contes), le DeliveryManager doit se composer du Graph (le réseau) et de l'Algorithm (Dijkstra) pour fonctionner.

Lignes Directrices de Modélisation (Composition) :

- Le Gestionnaire : Créez une classe principale **DeliveryManager** ou **LogisticsSystem** qui représente le cœur de l'application.
- Composition : Cette classe doit être composée (doit contenir en tant qu'attributs privés) :
 - L'instance du Graphe (**Graph**) qui représente le réseau complet de la ville (chargé avec les Restaurant, Client, et StreetSegment).
 - L'instance de l'Algorithme (**DijkstraAlgorithm**) qui effectuera les calculs de chemin.
- Chargement du Réseau : Le **DeliveryManager** doit implémenter une méthode pour importer les données du réseau (lieux et trajets) depuis un fichier (Liste d'adjacence ou d'arêtes, comme vu en TP2) afin de construire le graphe.

Simulation d'une Commande et Traçabilité

Une fois le réseau chargé, le manager doit pouvoir traiter une commande simple, de sa prise en charge à sa livraison. La livraison est une séquence de deux trajets calculés de manière optimale.

Écho au TD 3, Section 4 (Membres de Classe) : Vous pourriez utiliser des attributs de classe (static) pour gérer des identifiants uniques de commandes ou des constantes de coût maximal, renforçant le concept d'éléments partagés.

Travail Demandé (Méthode Principale) :

1. La Fonction de Planification : Implémentez une méthode `planDelivery`(Depot start, Restaurant pickup, Client dropoff) dans le `DeliveryManager`.
2. Composition du Trajet : Cette méthode doit utiliser l'instance de `DijkstraAlgorithm` pour calculer successivement :
 - Le trajet optimal du Dépôt au Restaurant.
 - Le trajet optimal du Restaurant au Client.
3. Rapport de Livraison : Le système doit afficher un rapport clair et structuré (similaire au rôle de la méthode `print()` de l'interface `Racontable` du TD 3, Section 2) qui détaille :
 - Les deux séquences de lieux à suivre.
 - Le temps total estimé pour l'ensemble de la livraison.

Les bonnes pratiques

Ce TP3 constitue la deuxième séance d'un projet conçu sur cinq séances. Gardez à l'esprit que le prochain TP, le TP4, portera sur l'analyse de réseaux et étendra le cadre applicatif de l'entreprise `SafeLogistic`, elle-même filiale de la société mère `Safe Inc.`. La filiale chargée de l'analyse de réseaux de type graphe se nomme `SafeNetmedia`. Cette information doit vous encourager à penser votre code en termes de modularité et de réutilisabilité.

Lors de la quatrième séance du projet, vous devrez mettre en commun l'ensemble du code développé dans les TP2, TP3 et TP4, selon un scénario dépendant du cadre applicatif, en utilisant un outil de gestion du cycle de vie logiciel en Java (Maven, TP5). Vous aborderez ces notions lors des cours magistraux après les vacances de Noël. Enfin, lors du TP6 (dernière séance), vous devrez concevoir une interface graphique avec `JavaFX`.

À partir du TP3, créez une branche distincte pour chaque séance (ex. : `tp3_s2projet_dev`) afin de mesurer vos progrès, de simuler un "mode collaboratif Git" et d'éviter les conflits.

À chaque fin de séance, pensez à mettre à jour votre documentation, à committer et à pousser votre code.

Les dates importantes

Vous devez impérativement valider une version bêta de votre logiciel lors de la dernière séance de TP (TP6), **prévue la semaine du 12 janvier 2026.**

Le projet final devra être rendu le **30 janvier 2026.**